

Write a program that will read in two numbers in as strings. Your program should store the digits in the number as a sort linked list. Your program should then merge the two linked list together in sorted order into a new linked list without changing the original two linked list. Your merge algorithm needs to run in $O(n+m)$ time with n and m being the length of the two linked lists. Your merge algorithm must be recursive.

```
Enter a number8926498
Enter a number124970912740
The linked lists
2->4->6->8->8->9->9->nullptr
0->0->1->1->2->2->4->4->7->7->9->9->nullptr
Merged linked lists
0->0->1->1->2->2->2->4->4->4->6->7->7->8->8->9->9->9->9->nullptr
The original linked lists again
2->4->6->8->8->9->9->nullptr
0->0->1->1->2->2->4->4->7->7->9->9->nullptr
_
```

Given the definition for a node in a linked list,

```
struct node {  
    int data;  
    node *next;  
};
```

and given a linked list p with unique values which are sorted in ascending order and a linked list odd and even which are nullptrs write a function

```
node* odd_even(node *p, node *&odd, node *&even)
```

that separates p into two linked list, odd which is an ascending linked list of odd values and even which is an ascending linked list of even values.

- odd_even must be a **destructive** function meaning that no new nodes are allocated and the original list p cannot be reconstructed.

Examples

if p points to the linked list [1 3 5 6 8 9] then odd should be [1 3 5 9] and even should be [6 8] and p should be nullptr.

if p points to the linked list [6 14 18] then odd should be nullptr and even should be [6 14 18] and p should be nullptr.

Should be call like this

```
p = odd_even(p, odd, even)
```

Do not use arrays. Do not use vectors. Do not strings. Do not use globals. Do not use static variables. Do not use the STL.

Given the definition for a node in a linked list,

```
struct node {  
    char data;  
    node *next;  
};
```

and given a linked list p write a function

```
node* org (node *p);
```

this function should move the nodes around so that will the nodes that contain a uppercase character come before the nodes with a lowercase number.

- `org` must not allocate memory for new node. You must change the next pointers in all the nodes so that the list is in a different order.

Examples

if p points to the linked list [A b c C c a] p should be [A C b c c a] (note the order of the chars don't matter what is important is the uppercases come first).

Should be call like this

```
p = org(p);
```

Do not use arrays. Do not use vectors. Do not strings. Do not use globals. Do not use static variables. Do not use the STL.